JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ObjectOutputStream

java.lang.Object
    java.io.OutputStream
        java.io.ObjectOutputStream

All Implemented Interfaces:

Closeable, DataOutput, Flushable, ObjectOutput, ObjectStreamConstants, AutoCloseable

---

```
public class ObjectOutputStream
extends OutputStream
implements ObjectOutput, ObjectStreamConstants
```

An ObjectOutputStream writes primitive data types and graphs of Java objects to an OutputStream. The objects can be read (reconstituted) using an ObjectInputStream. Persistent storage of objects can be accomplished by using a file for the stream. If the stream is a network socket stream, the objects can be reconstituted on another host or in another process.

Only objects that support the java.io.Serializable interface can be written to streams. The class of each serializable object is encoded including the class name and signature of the class, the values of the object's fields and arrays, and the closure of any other objects referenced from the initial objects.

The method writeObject is used to write an object to the stream. Any object, including Strings and arrays, is written with writeObject. Multiple objects or primitives can be written to the stream. The objects must be read back from the corresponding ObjectInputstream with the same types and in the same order as they were written.

Primitive data types can also be written to the stream using the appropriate methods from DataOutput. Strings can also be written using the writeUTF method.

The default serialization mechanism for an object writes the class of the object, the class signature, and the values of all non-transient and non-static fields. References to other objects (except in transient or static fields) cause those objects to be written also. Multiple references to a single object are encoded using a reference sharing mechanism so that graphs of objects can be restored to the same shape as when the original was written.

For example to write an object that can be read by the example in ObjectInputStream:

```
     try (FileOutputStream fos = new FileOutputStream("t.tmp");
          ObjectOutputStream oos = new ObjectOutputStream(fos)) {
```

```
            oos.writeObject("Today");
            oos.writeObject(LocalDateTime.now());
        } catch (Exception ex) {
            // handle exception
        }
```

Serializable classes that require special handling during the serialization and deserialization process should implement methods with the following signatures:

```
    private void readObject(java.io.ObjectInputStream stream)
        throws IOException, ClassNotFoundException;
    private void writeObject(java.io.ObjectOutputStream stream)
        throws IOException;
    private void readObjectNoData()
        throws ObjectStreamException;
```

The method name, modifiers, return type, and number and type of parameters must match exactly for the method to be used by serialization or deserialization. The methods should only be declared to throw checked exceptions consistent with these signatures.

The writeObject method is responsible for writing the state of the object for its particular class so that the corresponding readObject method can restore it. The method does not need to concern itself with the state belonging to the object's superclasses or subclasses. State is saved by writing the individual fields to the ObjectOutputStream using the writeObject method or by using the methods for primitive data types supported by DataOutput.

Serialization does not write out the fields of any object that does not implement the java.io.Serializable interface. Subclasses of Objects that are not serializable can be serializable. In this case the non-serializable class must have a no-arg constructor to allow its fields to be initialized. In this case it is the responsibility of the subclass to save and restore the state of the non-serializable class. It is frequently the case that the fields of that class are accessible (public, package, or protected) or that there are get and set methods that can be used to restore the state.

Serialization of an object can be prevented by implementing writeObject and readObject methods that throw the NotSerializableException. The exception will be caught by the ObjectOutputStream and abort the serialization process.

Implementing the Externalizable interface allows the object to assume complete control over the contents and format of the object's serialized form. The methods of the Externalizable interface, writeExternal and readExternal,

are called to save and restore the objects state. When implemented by a class they can write and read their own state using all of the methods of ObjectOutput and ObjectInput. It is the responsibility of the objects to handle any versioning that occurs.

Enum constants are serialized differently than ordinary serializable or externalizable objects. The serialized form of an enum constant consists solely of its name; field values of the constant are not transmitted. To serialize an enum constant, ObjectOutputStream writes the string returned by the constant's name method. Like other serializable or externalizable objects, enum constants can function as the targets of back references appearing subsequently in the serialization stream. The process by which enum constants are serialized cannot be customized; any class-specific writeObject and writeReplace methods defined by enum types are ignored during serialization. Similarly, any serialPersistentFields or serialVersionUID field declarations are also ignored--all enum types have a fixed serialVersionUID of 0L.

Primitive data, excluding serializable fields and externalizable data, is written to the ObjectOutputStream in block-data records. A block data record is composed of a header and data. The block data header consists of a marker and the number of bytes to follow the header. Consecutive primitive data writes are merged into one block-data record. The blocking factor used for a block-data record will be 1024 bytes. Each block-data record will be filled up to 1024 bytes, or be written whenever there is a termination of block-data mode. Calls to the ObjectOutputStream methods writeObject, defaultWriteObject and writeFields initially terminate any existing block-data record.

Records are serialized differently than ordinary serializable or externalizable objects, see record serialization.

Since:

1.1

External Specifications

Java Object Serialization Specification

See Also:

`DataOutput`,
`ObjectInputStream`,
`Serializable`,
`Externalizable`,
Java Object Serialization Specification, Section 2, "Object Output Classes"

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
static class	ObjectOutputStream.PutFi	Provide programmatic access to the persistent fields to be written to ObjectOutput.

Field Summary

Fields declared in interface ObjectStreamConstants

baseWireHandle, PROTOCOL_VERSION_1, PROTOCOL_VERSION_2, SC_BLOCK_DATA, SC_ENUM, SC_EXTERNALIZABLE, SC_SERIALIZABLE, SC_WRITE_METHOD, SERIAL_FILTER_PERMISSION, STREAM_MAGIC, STREAM_VERSION, SUBCLASS_IMPLEMENTATION_PERMISSION, SUBSTITUTION_PERMISSION, TC_ARRAY, TC_BASE, TC_BLOCKDATA, TC_BLOCKDATALONG, TC_CLASS, TC_CLASSDESC, TC_ENDBLOCKDATA, TC_ENUM, TC_EXCEPTION, TC_LONGSTRING, TC_MAX, TC_NULL, TC_OBJECT, TC_PROXYCLASSDESC, TC_REFERENCE, TC_RESET, TC_STRING

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	ObjectOutputStream()	Provide a way for subclasses that are completely reimplementing ObjectOutputStream to not have to allocate private data just used by this implementation of ObjectOutputStream.
	ObjectOutputStream (OutputStream out)	Creates an ObjectOutputStream that writes to the specified OutputStream.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
protected void	**annotateClass**(Class<?> cl)	Subclasses may implement this method to allow class data to be stored in the stream.
protected void	**annotateProxyClass**(Class<?> cl)	Subclasses may implement this method to store custom data in the stream along with descriptors for dynamic proxy classes.
void	**close**()	Closes the stream.
void	**defaultWriteObject**()	Write the non-static and non-transient fields of the current class to this stream.
protected void	**drain**()	Drain any buffered data in ObjectOutputStream.
protected boolean	**enableReplaceObject**(boolean enable)	Enables the stream to do replacement of objects written to the stream.
void	**flush**()	Flushes the stream.
ObjectOutputStream.P	**putFields**()	Retrieve the object used to buffer persistent fields to be written to the stream.
protected Object	**replaceObject**(Object obj)	This method will allow trusted subclasses of ObjectOutputStream to substitute one object for another during serialization.
void	**reset**()	Reset will disregard the state of any objects already written to the stream.

void	**useProtocolVersion**(int version)	Specify stream protocol version to use when writing the stream.
void	**write**(byte[] buf)	Writes an array of bytes.
void	**write**(byte[] buf, int off, int len)	Writes a sub array of bytes.
void	**write**(int val)	Writes a byte.
void	**writeBoolean**(boolean val)	Writes a boolean.
void	**writeByte**(int val)	Writes an 8-bit byte.
void	**writeBytes**(String str)	Writes a String as a sequence of bytes.
void	**writeChar**(int val)	Writes a 16-bit char.
void	**writeChars**(String str)	Writes a String as a sequence of chars.
protected void	**writeClassDescriptor**(ObjectStreamClass desc)	Write the specified class descriptor to the ObjectOutputStream.
void	**writeDouble**(double val)	Writes a 64-bit double.
void	**writeFields**()	Write the buffered fields to the stream.
void	**writeFloat**(float val)	Writes a 32-bit float.
void	**writeInt**(int val)	Writes a 32-bit int.
void	**writeLong**(long val)	Writes a 64-bit long.
final void	**writeObject**(Object obj)	Write the specified object to the ObjectOutputStream.
protected void	**writeObjectOverride**(Object obj)	Method used by subclasses to override the default writeObject method.
void	**writeShort**(int val)	

		Writes a 16-bit short.
protected void	**writeStreamHeader**()	The writeStreamHeader method is provided so subclasses can append or prepend their own header to the stream.
void	**writeUnshared**(Object obj)	Writes an "unshared" object to the ObjectOutputStream.
void	**writeUTF**(String str)	Primitive data write of this String in modified UTF-8 format.

Methods declared in class OutputStream

nullOutputStream

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

ObjectOutputStream

```
public ObjectOutputStream(OutputStream out)
                   throws IOException
```

Creates an ObjectOutputStream that writes to the specified OutputStream. This constructor writes the serialization stream header to the underlying stream; callers may wish to flush the stream immediately to ensure that constructors for receiving ObjectInputStreams will not block when reading the header.

Parameters:

out - output stream to write to

Throws:

IOException - if an I/O error occurs while writing stream header

NullPointerException - if out is null

Since:

1.4

See Also:

`ObjectOutputStream()`, `putFields()`, `ObjectInputStream(InputStream)`

ObjectOutputStream

```
protected ObjectOutputStream()
                     throws IOException
```

Provide a way for subclasses that are completely reimplementing ObjectOutputStream to not have to allocate private data just used by this implementation of ObjectOutputStream.

Throws:

`IOException` - if an I/O error occurs while creating this stream

Method Details

useProtocolVersion

```
public void useProtocolVersion(int version)
                        throws IOException
```

Specify stream protocol version to use when writing the stream.

This routine provides a hook to enable the current version of Serialization to write in a format that is backwards compatible to a previous version of the stream format.

Every effort will be made to avoid introducing additional backwards incompatibilities; however, sometimes there is no other alternative.

Parameters:

`version` - use ProtocolVersion from java.io.ObjectStreamConstants.

Throws:

`IllegalStateException` - if called after any objects have been serialized.

`IllegalArgumentException` - if invalid version is passed in.

`IOException` - if I/O errors occur

Since:

1.2

See Also:

`ObjectStreamConstants.PROTOCOL_VERSION_1`,
`ObjectStreamConstants.PROTOCOL_VERSION_2`

writeObject

```
public final void writeObject(Object obj)
                       throws IOException
```

Write the specified object to the ObjectOutputStream. The class of the object, the signature of the class, and the values of the non-transient and non-static fields of the class and all of its supertypes are written. Default serialization for a class can be overridden using the writeObject and the readObject methods. Objects referenced by this object are written transitively so that a complete equivalent graph of objects can be reconstructed by an ObjectInputStream.

Exceptions are thrown for problems with the OutputStream and for classes that should not be serialized. All exceptions are fatal to the OutputStream, which is left in an indeterminate state, and it is up to the caller to ignore or recover the stream state.

Specified by:

`writeObject` in interface `ObjectOutput`

Parameters:

`obj` - the object to be written

Throws:

`InvalidClassException` - Something is wrong with a class used by serialization.

`NotSerializableException` - Some object to be serialized does not implement the java.io.Serializable interface.

`IOException` - Any exception thrown by the underlying OutputStream.

writeObjectOverride

```
protected void writeObjectOverride(Object obj)
                            throws IOException
```

Method used by subclasses to override the default writeObject method. This method is called by trusted subclasses of ObjectOutputStream that constructed ObjectOutputStream using the protected no-arg constructor. The subclass is expected to provide an override method with the modifier "final".

Parameters:

`obj` - object to be written to the underlying stream

Throws:

`IOException` - if there are I/O errors while writing to the underlying stream

Since:

1.2

See Also:

`ObjectOutputStream()`, `writeObject(Object)`

writeUnshared

```
public void writeUnshared(Object obj)
                   throws IOException
```

Writes an "unshared" object to the ObjectOutputStream. This method is identical to writeObject, except that it always writes the given object as a new, unique object in the stream (as opposed to a back-reference pointing to a previously serialized instance). Specifically:

- An object written via writeUnshared is always serialized in the same manner as a newly appearing object (an object that has not been written to the stream yet), regardless of whether or not the object has been written previously.
- If writeObject is used to write an object that has been previously written with writeUnshared, the previous writeUnshared operation is treated as if it were a write of a separate object. In other words, ObjectOutputStream will never generate back-references to object data written by calls to writeUnshared.

While writing an object via writeUnshared does not in itself guarantee a unique reference to the object when it is deserialized, it allows a single object to be defined multiple times in a stream, so that multiple calls to readUnshared by the receiver will not conflict. Note that the rules described above only apply to the base-level object written with writeUnshared, and not to any transitively referenced sub-objects in the object graph to be serialized.

Parameters:

`obj` - object to write to stream

Throws:

`NotSerializableException` - if an object in the graph to be serialized does not implement the Serializable interface

`InvalidClassException` - if a problem exists with the class of an object to be serialized

`IOException` - if an I/O error occurs during serialization

Since:

1.4

defaultWriteObject

```
public void defaultWriteObject()
                        throws IOException
```

Write the non-static and non-transient fields of the current class to this stream. This may only be called from the writeObject method of the class being serialized. It will throw the NotActiveException if it is called otherwise.

Throws:

`IOException` - if I/O errors occur while writing to the underlying `OutputStream`

putFields

```
public ObjectOutputStream.PutField putFields()
                                     throws IOException
```

Retrieve the object used to buffer persistent fields to be written to the stream. The fields will be written to the stream when writeFields method is called.

Returns:

an instance of the class Putfield that holds the serializable fields

Throws:

`IOException` - if I/O errors occur

Since:

1.2

writeFields

```
public void writeFields()
                  throws IOException
```

Write the buffered fields to the stream.

Throws:

`IOException` - if I/O errors occur while writing to the underlying stream

`NotActiveException` - Called when a classes writeObject method was not called to write the state of the object.

Since:

1.2

reset

```
public void reset()
           throws IOException
```

Reset will disregard the state of any objects already written to the stream. The state is reset to be the same as a new ObjectOutputStream. The current point in the stream is marked as reset so the corresponding ObjectInputStream will be reset at the same point. Objects previously written

to the stream will not be referred to as already being in the stream. They will be written to the stream again.

Throws:

`IOException` - if reset() is invoked while serializing an object.

annotateClass

```
protected void annotateClass(Class<?> cl)
                      throws IOException
```

Subclasses may implement this method to allow class data to be stored in the stream. By default this method does nothing. The corresponding method in ObjectInputStream is resolveClass. This method is called exactly once for each unique class in the stream. The class name and signature will have already been written to the stream. This method may make free use of the ObjectOutputStream to save any representation of the class it deems suitable (for example, the bytes of the class file). The resolveClass method in the corresponding subclass of ObjectInputStream must read and use any data or objects written by annotateClass.

Parameters:

`cl` - the class to annotate custom data for

Throws:

`IOException` - Any exception thrown by the underlying OutputStream.

annotateProxyClass

```
protected void annotateProxyClass(Class<?> cl)
                           throws IOException
```

Subclasses may implement this method to store custom data in the stream along with descriptors for dynamic proxy classes.

This method is called exactly once for each unique proxy class descriptor in the stream. The default implementation of this method in `ObjectOutputStream` does nothing.

The corresponding method in `ObjectInputStream` is `resolveProxyClass`. For a given subclass of `ObjectOutputStream` that overrides this method, the `resolveProxyClass` method in the corresponding subclass of `ObjectInputStream` must read any data or objects written by `annotateProxyClass`.

Parameters:

`cl` - the proxy class to annotate custom data for

Throws:

IOException - any exception thrown by the underlying OutputStream

Since:

1.3

See Also:

ObjectInputStream.resolveProxyClass(String[])

replaceObject

```
protected Object replaceObject(Object obj)
                        throws IOException
```

This method will allow trusted subclasses of ObjectOutputStream to substitute one object for another during serialization. Replacing objects is disabled until enableReplaceObject is called. The enableReplaceObject method checks that the stream requesting to do replacement can be trusted. The first occurrence of each object written into the serialization stream is passed to replaceObject. Subsequent references to the object are replaced by the object returned by the original call to replaceObject. To ensure that the private state of objects is not unintentionally exposed, only trusted streams may use replaceObject.

The ObjectOutputStream.writeObject method takes a parameter of type Object (as opposed to type Serializable) to allow for cases where non-serializable objects are replaced by serializable ones.

When a subclass is replacing objects it must ensure that either a complementary substitution must be made during deserialization or that the substituted object is compatible with every field where the reference will be stored. Objects whose type is not a subclass of the type of the field or array element abort the serialization by raising an exception and the object is not be stored.

This method is called only once when each object is first encountered. All subsequent references to the object will be redirected to the new object. This method should return the object to be substituted or the original object.

Null can be returned as the object to be substituted, but may cause NullPointerException in classes that contain references to the original object since they may be expecting an object instead of null.

Parameters:

obj - the object to be replaced

Returns:

the alternate object that replaced the specified one

Throws:

IOException - Any exception thrown by the underlying OutputStream.

enableReplaceObject

```
protected boolean enableReplaceObject(boolean enable)
```

Enables the stream to do replacement of objects written to the stream. When enabled, the replaceObject(Object) method is called for every object being serialized.

Parameters:

enable - true for enabling use of replaceObject for every object being serialized

Returns:

the previous setting before this method was invoked

writeStreamHeader

```
protected void writeStreamHeader()
                           throws IOException
```

The writeStreamHeader method is provided so subclasses can append or prepend their own header to the stream. It writes the magic number and version to the stream.

Throws:

IOException - if I/O errors occur while writing to the underlying stream

writeClassDescriptor

```
protected void writeClassDescriptor(ObjectStreamClass desc)
                              throws IOException
```

Write the specified class descriptor to the ObjectOutputStream. Class descriptors are used to identify the classes of objects written to the stream. Subclasses of ObjectOutputStream may override this method to customize the way in which class descriptors are written to the serialization stream. The corresponding method in ObjectInputStream, readClassDescriptor, should then be overridden to reconstitute the class descriptor from its custom stream representation. By default, this method writes class descriptors according to the format defined in the *Java Object Serialization Specification*.

Note that this method will only be called if the ObjectOutputStream is not using
the old serialization stream format (set by calling ObjectOutputStream's useProtocolVersion method). If this serialization stream is using the old

format (PROTOCOL_VERSION_1), the class descriptor will be written internally in a manner that cannot be overridden or customized.

Parameters:

desc - class descriptor to write to the stream

Throws:

IOException - If an I/O error has occurred.

Since:

1.3

External Specifications

Java Object Serialization Specification

See Also:

ObjectInputStream.readClassDescriptor(),
useProtocolVersion(int),
ObjectStreamConstants.PROTOCOL_VERSION_1

write

```
public void write(int val)
           throws IOException
```

Writes a byte. This method will block until the byte is actually written.

Specified by:

write in interface DataOutput

Specified by:

write in interface ObjectOutput

Specified by:

write in class OutputStream

Parameters:

val - the byte to be written to the stream

Throws:

IOException - If an I/O error has occurred.

write

```
public void write(byte[] buf)
           throws IOException
```

Writes an array of bytes. This method will block until the bytes are actually written.

Specified by:

write in interface DataOutput

Specified by:

write in interface ObjectOutput

Overrides:

write in class OutputStream

Parameters:

buf - the data to be written

Throws:

IOException - If an I/O error has occurred.

See Also:

OutputStream.write(byte[], int, int)

write

```
public void write(byte[] buf,
                  int off,
                  int len)
           throws IOException
```

Writes a sub array of bytes.

Specified by:

write in interface DataOutput

Specified by:

write in interface ObjectOutput

Overrides:

write in class OutputStream

Parameters:

buf - the data to be written

off - the start offset in the data

len - the number of bytes that are written

Throws:

IOException - if an I/O error occurs. In particular, an IOException is thrown if the output stream is closed.

IndexOutOfBoundsException - If off is negative, len is negative, or len is greater than b.length - off

flush

```
public void flush()
           throws IOException
```

Flushes the stream. This will write any buffered output bytes and flush through to the underlying stream.

Specified by:

`flush` in interface `Flushable`

Specified by:

`flush` in interface `ObjectOutput`

Overrides:

`flush` in class `OutputStream`

Throws:

`IOException` - if an I/O error occurs.

drain

```
protected void drain()
              throws IOException
```

Drain any buffered data in ObjectOutputStream. Similar to flush but does not propagate the flush to the underlying stream.

Throws:

`IOException` - if I/O errors occur while writing to the underlying stream

close

```
public void close()
           throws IOException
```

Closes the stream. This method must be called to release any resources associated with the stream.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Specified by:

`close` in interface `ObjectOutput`

Overrides:

`close` in class `OutputStream`

Throws:

IOException - If an I/O error has occurred.

writeBoolean

```
public void writeBoolean(boolean val)
                  throws IOException
```

Writes a boolean.

Specified by:

writeBoolean in interface DataOutput

Parameters:

val - the boolean to be written

Throws:

IOException - if I/O errors occur while writing to the underlying stream

writeByte

```
public void writeByte(int val)
               throws IOException
```

Writes an 8-bit byte.

Specified by:

writeByte in interface DataOutput

Parameters:

val - the byte value to be written

Throws:

IOException - if I/O errors occur while writing to the underlying stream

writeShort

```
public void writeShort(int val)
                throws IOException
```

Writes a 16-bit short.

Specified by:

writeShort in interface DataOutput

Parameters:

val - the short value to be written

Throws:

IOException - if I/O errors occur while writing to the underlying stream

writeChar

```
public void writeChar(int val)
               throws IOException
```

Writes a 16-bit char.

Specified by:

`writeChar` in interface `DataOutput`

Parameters:

`val` - the char value to be written

Throws:

`IOException` - if I/O errors occur while writing to the underlying stream

writeInt

```
public void writeInt(int val)
              throws IOException
```

Writes a 32-bit int.

Specified by:

`writeInt` in interface `DataOutput`

Parameters:

`val` - the integer value to be written

Throws:

`IOException` - if I/O errors occur while writing to the underlying stream

writeLong

```
public void writeLong(long val)
               throws IOException
```

Writes a 64-bit long.

Specified by:

`writeLong` in interface `DataOutput`

Parameters:

`val` - the long value to be written

Throws:

`IOException` - if I/O errors occur while writing to the underlying stream

writeFloat

```
public void writeFloat(float val)
                throws IOException
```

Writes a 32-bit float.

Specified by:

writeFloat in interface DataOutput

Parameters:

val - the float value to be written

Throws:

IOException - if I/O errors occur while writing to the underlying stream

writeDouble

```
public void writeDouble(double val)
                 throws IOException
```

Writes a 64-bit double.

Specified by:

writeDouble in interface DataOutput

Parameters:

val - the double value to be written

Throws:

IOException - if I/O errors occur while writing to the underlying stream

writeBytes

```
public void writeBytes(String str)
                throws IOException
```

Writes a String as a sequence of bytes.

Specified by:

writeBytes in interface DataOutput

Parameters:

str - the String of bytes to be written

Throws:

IOException - if I/O errors occur while writing to the underlying stream

writeChars

```
public void writeChars(String str)
                throws IOException
```

Writes a String as a sequence of chars.

Specified by:

`writeChars` in interface `DataOutput`

Parameters:

`str` - the String of chars to be written

Throws:

`IOException` - if I/O errors occur while writing to the underlying stream

writeUTF

```
public void writeUTF(String str)
              throws IOException
```

Primitive data write of this String in modified UTF-8 format. Note that there is a significant difference between writing a String into the stream as primitive data or as an Object. A String instance written by writeObject is written into the stream as a String initially. Future writeObject() calls write references to the string into the stream.

Specified by:

`writeUTF` in interface `DataOutput`

Parameters:

`str` - the String to be written

Throws:

`IOException` - if I/O errors occur while writing to the underlying stream

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.